

Week 2 Design Report — Taxi Demand API on GKE

Arnav Mahale · AIPI 561 Operationalizing AI · Summer 2026

I deployed the pre-trained LightGBM taxi-demand FastAPI service to a 2-node GKE cluster (`n1-standard-2` , `autoscale 2-5` , `us-central1-a`) with GitHub Actions handling build and deploy on every push to `main` . The operational choices below are the ones that mattered.

Replicas + rolling updates. Two replicas behind a `LoadBalancer` Service so the deployment can absorb a pod restart or a rollout without user-visible downtime, and so the 10+ concurrent-request target is easily met. The rolling-update policy (`maxSurge: 1` , `maxUnavailable: 1`) keeps at least one pod serving traffic during every deploy without doubling resource usage on a 2-node cluster.

Resource requests vs limits — 512m / 1Gi requested, 1000m / 3Gi limit. Requests are tuned so two pods schedule comfortably on each `n1-standard-2` node alongside the init container and node overhead. Limits give the pod headroom for LightGBM's cold-start memory spike when the model + parquet are loaded. Intentionally under-utilized headroom is the price of reliability — the 2025 Kubernetes Benchmark reports 10% CPU / 23% memory averages industry-wide for the same reason.

Readiness vs liveness probes, deliberately different. Both hit `/health` , but the readiness probe checks fast (`initialDelaySeconds 20` , `period 10` , `fail 3`) so a struggling pod is pulled from the LB pool immediately; the liveness probe is slow (`initialDelaySeconds 60` , `period 20` , `fail 5`) so a still-warming pod isn't killed mid-load. This separation avoids the classic readiness/liveness restart-cascade.

Two-tier data loading. The small lookup files (`taxi_zones.geojson` , `zone_hour_avg_fare.parquet` , `taxi_zone_lookup.csv`) are baked into the Docker image for availability; the 74 MB demand parquet and 11 MB LightGBM model live in `gs://ops-ai-arnavmahale-data` and are pulled by an `initContainer` on every pod startup. A model swap is therefore a GCS upload + `kubectl rollout restart` — no image rebuild required.

CI/CD with immutable artifacts. `ci.yml` runs tests + a sanity Docker build on every push and PR. `cd.yml` runs only on `main` : it authenticates to GCP via the `GCP_SA_KEY` GitHub Secret, builds the image with `docker/build-push-action` , pushes both `:latest` and `:${{ github.sha }}` tags to Artifact Registry, updates the GKE deployment with `kubectl set image` , and waits on `kubectl rollout status` so a failed rollout fails the workflow. The SHA tag is the rollback story: every deploy is traceable to an exact commit and reversible in one `kubectl set image` command.

Auth and secrets. The `github-actions` service account holds four scoped roles (`container.developer` , `artifactregistry.writer` , `artifactregistry.reader` , `storage.objectViewer`). Its key is stored only in the `GCP_SA_KEY` GitHub Secret and in a Kubernetes `gcs-sa-key` Secret that's mounted into the init container; it is git-ignored locally and never committed. A separate `docker-registry` -type Kubernetes secret (`artifact-registry-secret`) lets pods pull the image from Artifact Registry.

Cleanup. GKE control plane + 2 worker nodes + LoadBalancer $\approx$ \$0.19/hr. The cluster and Artifact Registry repository are deleted immediately after grading screenshots are captured; the GCS bucket is retained for Week 3.